

PORTADA

Técnicas de Programación

Unidad 7 — Desarrollo Web: Funciones de Orden Superior y Fetch

Universidad de Antioquia · Ingeniería de Sistemas · 2508307



TRANSICIÓN

De manipular el DOM a transformar datos

En la sesión anterior aprendiste los fundamentos de JavaScript — variables, tipos, funciones — y a usarlos para **manipular el DOM**: seleccionar elementos, escuchar eventos, actualizar la página en respuesta a lo que hace el usuario. Ese JavaScript vivía todo dentro del navegador, trabajando con datos que ya estaban ahí.

Hoy das dos pasos más: primero, una forma más expresiva de **transformar colecciones de datos** con funciones de orden superior (`map` , `filter` , `reduce`); segundo, cómo pedirle esos datos a un servidor remoto con `fetch` , cuando no están en la página todavía.

CONCEPTO

Arrays en JavaScript: un repaso rápido

Ya conoces los arreglos desde Java (Unidad 0): una colección ordenada de valores accesibles por índice. En JavaScript funcionan igual en lo esencial, pero con sintaxis más flexible — no se declara el tipo ni el tamaño de antemano.

```
const precios = [12000, 8500, 21000];

console.log(precios[0]); // 12000 – acceso por índice
precios.push(15000);     // agrega un elemento al final
console.log(precios.length); // 4 – tamaño actual del array
```

Con esa base ya puedes leer y modificar un array elemento por elemento con un `for`, tal como en Java. Lo que viene ahora es una forma distinta de recorrerlo: en vez de un ciclo explícito, pasarle una función al propio array.

CONCEPTO

¿Qué es una función de orden superior?

Una **función de orden superior** (*higher-order function*) es una función que recibe otra función como argumento, la retorna, o ambas cosas. En vez de escribir un ciclo que describe **cómo** recorrer un array paso a paso, le entregas al array una función que describe **qué** hacer con cada elemento — y es el array quien se encarga del recorrido.

Los arrays de JavaScript traen tres funciones de orden superior centrales para transformar datos: `map()` , `filter()` y `reduce()` . Las tres reciben una función como argumento y ninguna modifica el array original.

CONCEPTO

map() : transforma cada elemento

`map()` recorre el array y aplica la función recibida a **cada elemento**, devolviendo un **array nuevo** del mismo tamaño, con los valores transformados. El array original queda intacto.

```
const precios = [12000, 8500, 21000];

const precioConIVA = precios.map(function (p) {
  return p * 1.19;
});

console.log(precioConIVA); // [14280, 10115, 24990]
console.log(precios);      // [12000, 8500, 21000] – sin cambios
```

CONCEPTO

`filter()` : selecciona los que cumplen una condición

`filter()` recorre el array y aplica una función que retorna `true` o `false` para cada elemento. El resultado es un array nuevo, que solo conserva los elementos donde la función devolvió `true` — puede ser más corto que el original, nunca más largo.

```
const precios = [12000, 8500, 21000, 5000];

const caros = precios.filter(function (p) {
  return p > 10000;
});

console.log(caros); // [12000, 21000]
```

CONCEPTO

reduce() : acumula en un solo valor

`reduce()` recorre el array y va combinando sus elementos en **un único valor** de salida (una suma, un conteo, un objeto armado a partir de la lista). Recibe una función con dos parámetros — el **acumulador** y el elemento actual — y un **valor inicial** del acumulador.

```
const carrito = [12000, 8500, 21000];

const total = carrito.reduce(function (acumulado, precio) {
  return acumulado + precio;
}, 0); // 0 es el valor inicial del acumulador

console.log(total); // 41500
```

CONCEPTO · COMPARACIÓN

La misma idea, dos lenguajes: Streams de Java

Esto no es nuevo del todo. En la Unidad 2 (semana 6, "Profundización: Lambdas y Streams en Java") ya viste

`.stream().filter().map()` sobre una `List` en Java. Es exactamente el mismo concepto de **programación funcional** — transformar datos encadenando funciones, sin ciclos explícitos ni mutar la colección original — expresado con otra sintaxis.

JavaScript (array)	Java (Stream)
<pre>const total = carrito .filter(p => p > 10000) .map(p => p * 1.19) .reduce((a, p) => a + p, 0);</pre>	<pre>double total = carrito.stream() .filter(p -> p > 10000) .map(p -> p * 1.19) .reduce(0.0, Double::sum);</pre>

Mismo paradigma funcional, dos sintaxis: filtrar, transformar y acumular sin ciclos ni variables mutables intermedias.

CONCEPTO

Callbacks, arrow functions y encadenamiento

La función que le pasas a `map` / `filter` / `reduce` se llama **callback**: una función que otra función va a invocar por ti, no tú directamente. Casi nunca se escribe con `function` como en los ejemplos anteriores — en la práctica se usa la sintaxis compacta de **arrow function**.

```
// Forma larga (como en los ejemplos anteriores)
precios.map(function (p) { return p * 1.19; });

// Arrow function equivalente – más común en código real
precios.map(p => p * 1.19);

// Encadenamiento: cada método retorna un array nuevo,
// así que se puede seguir llamando el siguiente método sobre él
const totalCaros = precios
  .filter(p => p > 10000)
  .map(p => p * 1.19)
  .reduce((acc, p) => acc + p, 0);
```

TRANSICIÓN

¿Y si los datos no están en el array todavía?

Hasta ahora `precios` ya existía en memoria. Pero lo normal en una aplicación web es que esos datos vivan en un **servidor remoto**, y haya que pedirlos por la red — como cuando el navegador carga una página. El problema: una petición de red puede tardar milisegundos o varios segundos, y JavaScript corre en **un solo hilo**. Si esperara "congelado" la respuesta, toda la página dejaría de responder mientras tanto: no se podría hacer clic en nada, ni siquiera hacer scroll.

Por eso JavaScript maneja este tipo de operaciones de forma **asíncrona**: lanza la petición, sigue ejecutando el resto del programa, y retoma el resultado cuando la red responde.

CONCEPTO

Promesas: un valor que todavía no existe

Una `Promise` es un objeto que representa el resultado **eventual** de una operación asíncrona (como una petición de red). Tiene tres estados posibles:

Estado	Significa
pendiente (<i>pending</i>)	La operación todavía no terminó
resuelta (<i>fulfilled</i>)	Terminó con éxito, ya hay un valor disponible
rechazada (<i>rejected</i>)	Terminó con error

```
promesa
  .then(resultado => console.log(resultado)) // se ejecuta si se resuelve
  .catch(error => console.log(error));      // se ejecuta si se rechaza
```

CONCEPTO

async / await : la misma promesa, más legible

`async / await` no es un mecanismo distinto a las promesas — es **azúcar sintáctico** sobre ellas: código que se lee de arriba a abajo como si fuera síncrono, aunque por debajo sigue siendo asíncrono. Una función marcada `async` puede usar `await` para "pausar" hasta que una promesa se resuelva, sin bloquear el resto de la página.

```
// Con .then()
obtenerDatos().then(datos => {
  console.log(datos);
});

// Equivalente con async/await
async function mostrarDatos() {
  const datos = await obtenerDatos();
  console.log(datos);
}
```

CONCEPTO

fetch() : pedir datos a una API

`fetch(url)` hace una petición **GET** a la URL indicada y retorna una `Promise` — retoma el método HTTP que ya viste en la primera sesión de esta unidad. Esa promesa se resuelve con un objeto `Response` ; para obtener el cuerpo como datos usables hay que llamar a `.json()` , que también retorna otra promesa.

```
fetch("https://api.ejemplo.com/productos")
  .then(response => response.json())
  .then(data => {
    console.log(data); // el array de productos ya como objetos JS
  });

// Equivalente con async/await
async function obtenerProductos() {
  const response = await fetch("https://api.ejemplo.com/productos");
  const data = await response.json();
  console.log(data);
}
```

CONCEPTO

JSON, el mismo formato en JavaScript y en Java

En la Unidad 4 (sesión "Gestión de Archivos: JSON") ya viste la sintaxis JSON y cómo Gson convierte texto JSON en objetos Java (deserializar) y objetos Java en texto JSON (serializar). En JavaScript el formato es idéntico — de hecho JSON nació de la notación de objetos de JavaScript — y las conversiones equivalentes son `JSON.parse()` y `JSON.stringify()` .

Operación	Java (Gson)	JavaScript
Texto JSON → objeto	<code>gson.fromJson(texto, Producto.class)</code>	<code>JSON.parse(texto)</code>
Objeto → texto JSON	<code>gson.toJson(producto)</code>	<code>JSON.stringify(producto)</code>

`fetch(...).then(res => res.json())` ya hace este `JSON.parse()` por dentro — por eso el resultado llega como objeto JS listo para usar.

CONTRA EJEMPLO

fetch sin manejo de errores: la falla en silencio

```
fetch("https://api.ejemplo.com/productos")
  .then(response => response.json())
  .then(data => mostrarProductos(data));
// si el servidor responde 404 o 500, ¡esto no lo detecta!
```

Este es el mismo problema que viste con las excepciones "tragadas" en Java, pero con un matiz importante: **fetch solo rechaza su promesa por fallas de red** (sin conexión, DNS caído). Si el servidor responde con un error HTTP (404, 500), **fetch** lo considera una respuesta válida — `response.json()` se ejecuta igual, sobre un cuerpo de error, y la página sigue como si nada.

CONCEPTO

Manejo correcto de errores en fetch

Dos piezas cubren los dos tipos de falla: `.catch()` captura errores de red, y verificar `response.ok` (verdadero solo si el código de estado HTTP está entre 200 y 299) detecta errores del servidor.

```
fetch("https://api.ejemplo.com/productos")
  .then(response => {
    if (!response.ok) {
      throw new Error("Error del servidor: " + response.status);
    }
    return response.json();
  })
  .then(data => mostrarProductos(data))
  .catch(error => console.log("Fallo la petición: " + error.message));
```


INTERACCIÓN

Consuman una API pública

En parejas: escriban un `fetch` hacia un endpoint público de ejemplo (ej. `https://jsonplaceholder.typicode.com/users`), encadenen `.then()` para convertir la respuesta a JSON, y usen `map()` para transformar cada elemento (por ejemplo, quedarse solo con el nombre y el correo de cada usuario) antes de mostrarlo con `console.log()` .

8 minutos · incluyan manejo de errores con `.catch()` y verificando `response.ok`

SÍNTESIS

Lo que hay que llevarse de hoy

1. Una función de orden superior recibe otra función como argumento; `map`, `filter` y `reduce` transforman, seleccionan y acumulan arrays sin ciclos explícitos ni mutar el original.
2. Es el mismo paradigma funcional que `.stream().map().filter().reduce()` en Java (Unidad 2) — dos sintaxis para la misma idea.
3. JavaScript es asíncrono para no bloquear la página mientras espera la red; las `Promise` y `async / await` son las herramientas para trabajar con eso.
4. `fetch()` pide datos a una API; `response.ok` y `.catch()` son obligatorios para no dejar errores del servidor pasar en silencio.



CIERRE

Antes de la próxima sesión

- Practicar el encadenamiento `filter().map().reduce()` sobre un array propio, comparando mentalmente con el equivalente en Streams de Java.
- Probar un `fetch` a `https://jsonplaceholder.typicode.com` con manejo de errores completo (`response.ok` + `.catch()`).

Próxima sesión: Spring Boot — el backend que expone la API que acabamos de aprender a consumir con `fetch` .

